

Ampliando: CLI para creadores de contenido

Automatización avanzada con `opencode run`

JA Palazón Ferrando

2026-06-22

Contenidos

1	¿Por qué usar CLI?	2
1.1	Ejemplo práctico	2
2	Conceptos básicos de la shell	3
2.1	Línea de comandos (<i>shell</i>)	3
2.2	Entrada y salida estándar	3
2.3	Redirección	3
2.4	Tuberías (<i>pipes</i>)	3
2.5	Ejercicios básicos	3
3	Redirección y tuberías	4
3.1	Ejercicio 1 — Probar ambas formas	4
4	Guardar salida en archivos	4
4.1	Ejercicio 2 — Traducir y guardar	4
4.2	Ejercicio 3 — Acumular resultados	5
5	Procesar varios archivos	5
5.1	Ejercicio 4 — Revisar todos los archivos	5
6	Scripts de automatización	5
6.1	Ejercicio 5 — Crear un script de revisión	5
7	Usar agentes desde CLI	6
7.1	Ejercicio 6 — Invocar un agente	6
8	Buenas prácticas	6
9	Ideas para automatizar	6
10	Referencias	7

i Has visto lo básico en el paso 6

Este documento amplía lo que viste en el paso 6. Si no lo has visto aún, te recomiendo empezar por allí.

💡 ¿Para quién es esta guía?

Esta guía amplía la información sobre la CLI de OpenCode para quienes quieren automatizar tareas repetitivas. **No necesitas ser programador**, pero sí aprender algunos conceptos básicos de la *shell*. Los ejemplos usan comandos simples que puedes copiar y pegar directamente.

1. ¿Por qué usar CLI?

Ventaja	Chat interactivo	CLI
Velocidad	Lento (conversación)	Rápido (directo)
Precisión	El modelo decide	Tú decides exactamente
Escalabilidad	Limitada (1 archivo)	Sin límite
Automatización	No posible	Scripts programables
Repetibilidad	Variable	Consistente
Coste	Mayor (historial)	Menor (sin contexto)
Ahorro de tokens	Historial acumulado	Sin historial

i Ahorro de tokens

En chat, cada mensaje incluye el historial completo de la conversación. En CLI, cada `opencode run` es independiente: solo procesa la instrucción actual. Esto reduce significativamente el consumo de tokens.

1.1. Ejemplo práctico

En chat (lento, paso a paso):

```
Tú: "Revisa ortografía de paso-01.qmd"
OpenCode: [procesa]
Tú: "Ahora paso-02.qmd"
OpenCode: [procesa]
Tú: "Y también paso-03.qmd"
...
```

En CLI (rápido, automatizado):

```
for f in paso-*.qmd; do
  opencode run "Revisa ortografía" @"$f"
done
```

i ¿Cuándo usar cada uno?

- **Chat**: explorar, aprender, tareas únicas, cuando no sabes exactamente qué quieres hacer
- **CLI**: lotes, automatización, tareas repetitivas, cuando sabes exactamente qué necesitas

2. Conceptos básicos de la shell

Antes de usar la CLI de OpenCode, es útil entender algunos conceptos fundamentales de la línea de comandos.

2.1. Línea de comandos (*shell*)

La *shell* es un programa que interpreta comandos de texto. En lugar de hacer clic en botones, escribes instrucciones directamente.

2.2. Entrada y salida estándar

Concepto	Significado	Ejemplo
Entrada estándar (<i>stdin</i>)	Por defecto, el teclado	Texto que escribes
Salida estándar (<i>stdout</i>)	Por defecto, la pantalla	Resultado del comando
Error (<i>stderr</i>)	Mensajes de error	“Archivo no encontrado”

2.3. Redirección

La redirección te permite controlar de dónde viene la entrada y adónde va la salida:

Símbolo	Función	Ejemplo
<	Entrada desde archivo	<code>opencode run "... " < archivo.md</code>
>	Salida a archivo (sobrescribe)	<code>> resultado.md</code>
>>	Salida a archivo (añade)	<code>>> informe.txt</code>

2.4. Tuberías (*pipes*)

La tubería `|` conecta la salida de un comando con la entrada del siguiente:

```
cat archivo.md | opencode run "Resume esto"
```

Analogía: piensa en la tubería como una cadena de producción. El resultado de un paso alimenta al siguiente.

2.5. Ejercicios básicos

Ejercicio 1 — Redirección de entrada:

```
echo "Hola mundo" > test.txt
opencode run "¿Qué dice este archivo?" < test.txt
```

Ejercicio 2 — Redirección de salida:

```
opencode run "Dime tres frutas" > frutas.txt
cat frutas.txt
```

Ejercicio 3 — Tubería:

```
echo "Python, R, JavaScript" | opencode run "¿Cuáles son estos lenguajes?"
```

i Recuerda: texto plano

Lo que pasa por una tubería es texto plano. Por eso Markdown es ideal: es legible tanto para ti como para OpenCode. Si le pasas un archivo `.qmd`, OpenCode recibirá el contenido sin formato.

3. Redirección y tuberías

Cuando trabajas con `opencode run`, tienes dos formas de pasar el contenido de un archivo:

Método	Sintaxis	Cuándo usarlo
Redirección <	<code>opencode run "... " < archivo</code>	Más directo, evita proceso extra
Tubería	<code>cat archivo opencode run "... "</code>	Cuando necesitas encadenar comandos

Analogía: piensa en la redirección como entregar un documento directamente a OpenCode. La tubería es como pasar el documento por varias manos antes de que llegue a OpenCode.

3.1. Ejercicio 1 — Probar ambas formas

```
opencode run "Resume este documento en 3 líneas" < paso-03.qmd
```

```
cat paso-03.qmd | opencode run "Resume este documento en 3 líneas"
```

Ambos comandos dan el mismo resultado. Usa `<` cuando sea posible: es más limpio y rápido.

4. Guardar salida en archivos

Puedes guardar el resultado de `opencode run` en un archivo para usarlo después:

Operador	Qué hace	Ejemplo
<code>></code>	Sobrescribe el archivo	<code>> resultado.md</code>
<code>>></code>	Añade al final del archivo	<code>>> informe.txt</code>

4.1. Ejercicio 2 — Traducir y guardar

```
opencode run "Traduce este documento al inglés" @paso-01.qmd > 01-agentes-en.md
```

El resultado se guarda en `01-agentes-en.md` sin mostrarlo en pantalla. Puedes abrirlo después para revisarlo.

4.2. Ejercicio 3 — Acumular resultados

```
opencode run "¿Qué errores hay en este archivo?" @paso-01.qmd > informe.txt
opencode run "¿Qué errores hay en este archivo?" @paso-02.qmd >> informe.txt
```

El primer comando crea `informe.txt`. El segundo añade los resultados del segundo archivo al final del mismo.

5. Procesar varios archivos

Si tienes varios archivos que necesitan la misma operación, puedes usar un bucle `for` de la *shell*:

5.1. Ejercicio 4 — Revisar todos los archivos

```
for f in *.qmd; do
  opencode run "Revisa la ortografía de este documento" @"$f" >> revisiones.txt
done
```

Este comando procesa todos los archivos `.qmd` del directorio y acumula las revisiones en `revisiones.txt`.

Consejo: si procesas muchos archivos, añade `sleep 1` dentro del bucle para evitar límites de tasa:

```
for f in *.qmd; do
  opencode run "Revisa la ortografía" @"$f" >> revisiones.txt
  sleep 1
done
```

6. Scripts de automatización

Para tareas que repites frecuentemente, puedes crear un script de shell que ejecuta varios comandos en secuencia.

6.1. Ejercicio 5 — Crear un script de revisión

Crea un archivo `revisar-todo.sh` en la raíz del proyecto:

```
#!/bin/bash
echo "=== Revisión automática del curso ===" > informe.txt
for f in *.qmd; do
  echo "--- Revisando $f ---" >> informe.txt
  opencode run "Revisa el tono y estilo de este documento" @"$f" >> informe.txt
  echo "" >> informe.txt
done
echo "=== Revisión completa ===" >> informe.txt
```

Hazlo ejecutable y pruébalo:

```
chmod +x revisar-todo.sh
./revisar-todo.sh
```

Abre `informe.txt` para ver los resultados de todos los archivos procesados.

¿Qué es `chmod +x`?

El comando `chmod +x revisar-todo.sh` hace que el archivo sea ejecutable. Sin esto, el sistema no te dejaría ejecutarlo directamente con `./revisar-todo.sh`.

No te agobies

Si no sabes cómo escribir un script, ¡pide ayuda a OpenCode! Escríbele algo como: “Crea un script que revise la ortografía de todos los archivos .qmd”. OpenCode generará el código por ti.

7. Usar agentes desde CLI

Si tienes agentes personalizados configurados en `.opencode/agents/`, puedes invocarlos desde la terminal:

7.1. Ejercicio 6 — Invocar un agente

```
opencode run "@revisor-estilo Revisa este documento" @paso-04.qmd
```

Funciona igual que desde el chat: el agente recibe la instrucción y procesa el archivo que le indiques.

8. Buenas prácticas

Práctica	Por qué es importante
Revisa siempre la salida	El CLI no es interactivo; no puedes corregir sobre la marcha
Usa archivos pequeños	Para lotes grandes, procesa de uno en uno
Mide los tiempos	Cada ejecución consume tokens; calcula el coste antes de lanzar procesos masivos
Combina con git	Haz un commit antes de procesar, por si el resultado no es el esperado

9. Ideas para automatizar

Tarea	Comando
Revisar ortografía de todos los .md	<code>for f in *.md; do opencode run "Revisa ortografía" @"\$f"; done</code>
Traducir un lote al inglés	<code>for f in *.qmd; do opencode run "Traduce al inglés" @"\$f" > "\${f%.qmd}-en.qmd"; done</code>
Generar resumen ejecutivo	<code>opencode run "Resume en 5 líneas" @informe-final.md > resumen.md</code>

Tarea	Comando
Unificar tono en varios docs	<code>opencode run "Aplica tono corporativo" @doc1.md @doc2.md</code>

¿Necesitas más ejemplos? Prueba a pedirle a OpenCode que te ayude a crear scripts personalizados para tu flujo de trabajo. La CLI es muy flexible y se adapta a casi cualquier necesidad de automatización.

10. Referencias

- [Documentación oficial de OpenCode](#)
- [GNU Bash Reference Manual](#)
- [Explainshell](#) — explica comandos gráficamente
- [Shell Check](#) — analiza scripts de shell

Curso completo disponible en: palazon.github.io/ocCursoAvanzado